

Algorithme de Boosting

— université
— lumière
— LYON 2



1 Motivations

2 Illustration concrète

3 Algorithme du boosting

4 Application : algorithme de Hedge

5 Boosting en classification

Considérons un problème concret de classification :

- On souhaite prédire si un patient est **à risque** ou **non à risque** à partir de variables médicales simples.

Considérons un problème concret de classification :

- On souhaite prédire si un patient est **à risque** ou **non à risque** à partir de variables médicales simples.

Exemple

On dispose de plusieurs modèles simples :

1. un arbre basé sur l'âge, le sexe et le type de comorbidité.
2. une règle basée sur la tension artérielle,
3. un score basé sur des biomarqueurs.

Considérons un problème concret de classification :

- On souhaite prédire si un patient est **à risque** ou **non à risque** à partir de variables médicales simples.

Exemple

On dispose de plusieurs modèles simples :

1. un arbre basé sur l'âge, le sexe et le type de comorbidité.
2. une règle basée sur la tension artérielle,
3. un score basé sur des biomarqueurs.

- Chaque modèle est **utile mais imparfait** :
 - ✗ il se trompe parfois,
 - ✗ et on ne sait pas à l'avance lequel est le plus fiable.

Considérons un problème concret de classification :

- On souhaite prédire si un patient est **à risque** ou **non à risque** à partir de variables médicales simples.

Exemple

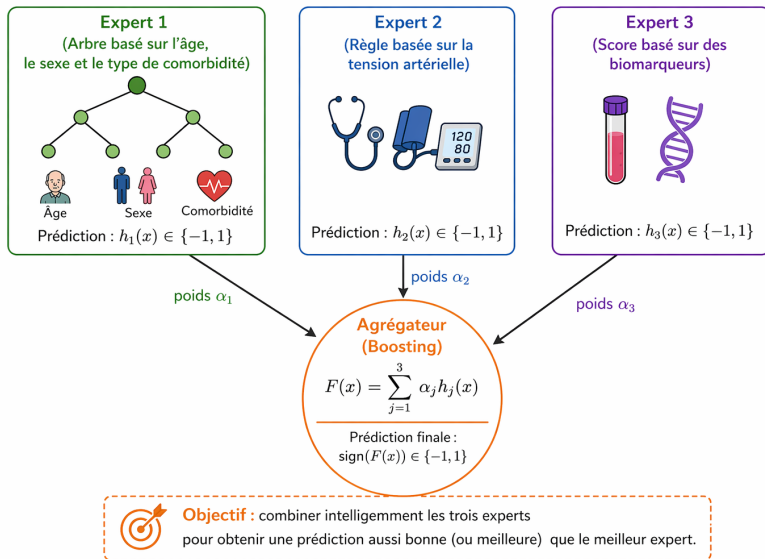
On dispose de plusieurs modèles simples :

1. un arbre basé sur l'âge, le sexe et le type de comorbidité.
2. une règle basée sur la tension artérielle,
3. un score basé sur des biomarqueurs.

- Chaque modèle est **utile mais imparfait** :
 - ✗ il se trompe parfois,
 - ✗ et on ne sait pas à l'avance lequel est le plus fiable.
- Question naturelle :

peut-on combiner intelligemment ces modèles pour faire mieux ?

⇒ Plutôt que choisir un seul modèle, on peut construire une **combinaison adaptative de plusieurs modèles simples** qui apprend progressivement de leurs erreurs.



Motivation

Le **Boosting** consiste à construire une **combinaison séquentielle de prédicteurs simples** afin d'obtenir une performance proche (ou meilleure) que celle du **meilleur expert disponible**, sans savoir à l'avance lequel est le meilleur.

Motivation

Le **Boosting** consiste à construire une **combinaison séquentielle de prédicteurs simples** afin d'obtenir une performance proche (ou meilleure) que celle du **meilleur expert disponible**, sans savoir à l'avance lequel est le meilleur.

Idée centrale :

- On dispose de N experts (arbres, modèles simples, règles).
- Chaque expert fait parfois des erreurs.
- On agrège leurs prédictions avec des **poids adaptatifs**.

Motivation

Le **Boosting** consiste à construire une **combinaison séquentielle de prédicteurs simples** afin d'obtenir une performance proche (ou meilleure) que celle du **meilleur expert disponible**, sans savoir à l'avance lequel est le meilleur.

Idée centrale :

- On dispose de N experts (arbres, modèles simples, règles).
- Chaque expert fait parfois des erreurs.
- On agrège leurs prédictions avec des **poids adaptatifs**.

Objectif

Construire un prédicteur global du type **[Agrégateur $\approx$ Meilleur expert]**.

Voire même meilleur que le meilleur expert lui-même dans certaines situations!

Question fondatrice (années 1990): Peut-on transformer un classifieur **légèrement meilleur que le hasard** en un classifieur **arbitrairement performant** ?

Question fondatrice (années 1990): Peut-on transformer un classifieur **légèrement meilleur que le hasard** en un classifieur **arbitrairement performant** ?

Résultat théorique fondateur

- **Robert Schapire (1990)**

The strength of weak learnability

Montre qu'un ensemble de classifieurs faibles peut être combiné pour construire un classifieur fort.

Question fondatrice (années 1990): Peut-on transformer un classifieur **légèrement meilleur que le hasard** en un classifieur **arbitrairement performant** ?

Résultat théorique fondateur

- **Robert Schapire (1990)**

The strength of weak learnability

Montre qu'un ensemble de classifieurs faibles peut être combiné pour construire un classifieur fort.

Algorithme pratique : AdaBoost

- **Yoav Freund & Robert Schapire (1997)**

A decision-theoretic generalization of on-line learning and an application to boosting

Premier algorithme efficace de Boosting avec garantie théorique.

Question fondatrice (années 1990): Peut-on transformer un classifieur **légèrement meilleur que le hasard** en un classifieur **arbitrairement performant** ?

Résultat théorique fondateur

- **Robert Schapire (1990)**

The strength of weak learnability

Montre qu'un ensemble de classifieurs faibles peut être combiné pour construire un classifieur fort.

Algorithme pratique : AdaBoost

- **Yoav Freund & Robert Schapire (1997)**

A decision-theoretic generalization of on-line learning and an application to boosting

Premier algorithme efficace de Boosting avec garantie théorique.

Interprétation moderne : Gradient Boosting

- **Jerome Friedman (2001)**

Greedy Function Approximation: A Gradient Boosting Machine

Montre que le Boosting correspond à une procédure de **descente de gradient dans un espace de fonctions**.

Les acteurs du jeu

On considère une interaction entre :

- un **agrégateur**
- N **experts** (proposant chacun une prédiction)
- le **monde réel** (qui révèle la vraie réponse)

Les acteurs du jeu

On considère une interaction entre :

- un **agrégateur**
- N **experts** (proposant chacun une prédiction)
- le **monde réel** (qui révèle la vraie réponse)

Déroulement à l'instant t

Pour chaque étape de décision notée $t = 1, \dots, T$:

1. chaque expert prédit **oui** ou **non**
2. l'agrégateur combine ces avis
3. le vrai résultat est révélé
4. l'agrégateur subit une perte si erreur

Les acteurs du jeu

On considère une interaction entre :

- un **agrégateur**
- N **experts** (proposant chacun une prédiction)
- le **monde réel** (qui révèle la vraie réponse)

Déroulement à l'instant t

Pour chaque étape de décision notée $t = 1, \dots, T$:

1. chaque expert prédit **oui** ou **non**
2. l'agrégateur combine ces avis
3. le vrai résultat est révélé
4. l'agrégateur subit une perte si erreur

Objectif

Construire une stratégie d'agrégation telle que la perte cumulée

$$\sum_{t=1}^T \text{Loss}^{(t)}$$

soit proche de celle du **meilleur expert *a posteriori***.

Distribution sur les experts

On associe un poids à chaque expert :

$$w_1, \dots, w_N$$

Ces poids définissent une loi de probabilité :

$$p_n = \frac{w_n}{\sum_{i=1}^N w_i}$$

Interprétation :

- ✓ expert fiable $\Rightarrow$ poids élevé
- ✗ expert peu fiable $\Rightarrow$ poids faible

Mise à jour après observation du résultat

À chaque instant t :

$$w_n^{(t)} = w_n^{(t-1)} (1 - \varepsilon \text{res}_n^{(t)})$$

où

$$\text{res}_n^{(t)} = \begin{cases} 0 & \text{expert correct} \\ 1 & \text{expert incorrect} \end{cases}$$

correspond à la *response mismatch* du n -ième expert. Donc :

erreur $\Rightarrow$ diminution du poids

Rôle du paramètre d'apprentissage ε

Le paramètre $\varepsilon \in]0, 1[$ contrôle la pénalisation appliquée aux experts qui commettent une erreur.

↑ si ε est grand : les experts mauvais perdent rapidement de l'influence. *Attention, si ε trop proche de 1; l'algorithme devient très instable et sur-réagit aux erreurs ponctuelles : il risque d'abandonner trop vite un bon expert après une seule erreur.*

↓ si ε est petit : l'algorithme s'adapte plus progressivement.

⇒ ε règle la vitesse d'adaptation de l'agrégateur aux performances passées des experts : $w \leftarrow (1 - \varepsilon)w$.

Idée fondamentale du Boosting

Le modèle apprend progressivement une **distribution adaptative sur les experts** :

bons experts $\uparrow$ mauvais experts $\downarrow$

L'agrégateur se rapproche ainsi automatiquement du **meilleur expert au cours du temps**.

1 Motivations

2 Illustration concrète

3 Algorithme du boosting

4 Application : algorithme de Hedge

5 Boosting en classification

On reprend notre exemple des trois experts :

1. un arbre basé sur l'âge, le sexe et le type de comorbidité
2. une règle basée sur la tension artérielle
3. un score basé sur des biomarqueurs

Initialisation du Boosting

Au départ, aucun expert n'est privilégié :

$$w_1^{(0)} = w_2^{(0)} = w_3^{(0)} = 1$$

Donc :

$$p_1^{(0)} = p_2^{(0)} = p_3^{(0)} = \frac{1}{3}$$

Tous les experts ont la même influence.

On observe un patient.
Les experts donnent :

Expert	Prédiction
1	à risque
2	à risque
3	non à risque

Or, il s'avère que le vrai diagnostic est **à risque**. Donc :

- ✓ expert 1 : correct
- ✓ expert 2 : correct
- ✗ expert 3 : incorrect

Règle du Boosting :

$$w_n^{(1)} = w_n^{(0)} (1 - \varepsilon \text{res}_n^{(1)})$$

où la perte est définie comme telle :

Perte

$$\text{res}_n^{(1)} = \begin{cases} 0 & \text{si expert correct} \\ 1 & \text{sinon} \end{cases}$$

Donc : $w_1^{(1)} = 1$; $w_2^{(1)} = 1$; $w_3^{(1)} = 1 - \varepsilon$

Règle du Boosting :

$$w_n^{(1)} = w_n^{(0)} (1 - \varepsilon \text{res}_n^{(1)})$$

où la perte est définie comme telle :

Perte

$$\text{res}_n^{(1)} = \begin{cases} 0 & \text{si expert correct} \\ 1 & \text{sinon} \end{cases}$$

Donc : $w_1^{(1)} = 1$; $w_2^{(1)} = 1$; $w_3^{(1)} = 1 - \varepsilon$

Conséquence à l'étape $t = 1$

L'expert biomarqueur devient moins influent : il s'est trompé.

On le pénalise donc par un facteur multiplicatif $(1 - \varepsilon)$ avec $\varepsilon \in]0, 1[$.

La nouvelle loi devient :

$$p_n^{(1)} = \frac{w_n^{(1)}}{\sum_{i=1}^3 w_i^{(1)}}$$

Donc :

$$p_1^{(1)} > p_3^{(1)} ; \quad p_2^{(1)} > p_3^{(1)}$$

La nouvelle loi devient :

$$p_n^{(1)} = \frac{w_n^{(1)}}{\sum_{i=1}^3 w_i^{(1)}}$$

Donc :

$$p_1^{(1)} > p_3^{(1)} ; \quad p_2^{(1)} > p_3^{(1)}$$

Conséquence

Lors de la prochaine prédiction :

l'agrégateur fera davantage confiance aux experts fiables

On observe maintenant un nouveau patient.
Supposons :

Expert	Prédiction
1	non à risque
2	à risque
3	à risque

On observe maintenant un nouveau patient.
Supposons :

Expert	Prédiction
1	non à risque
2	à risque
3	à risque

Le vrai diagnostic est à risque. Donc :

On observe maintenant un nouveau patient.
Supposons :

Expert	Prédiction
1	non à risque
2	à risque
3	à risque

Le vrai diagnostic est **à risque**. Donc :

- ✗ expert 1 : incorrect
- ✓ expert 2 : correct
- ✓ expert 3 : correct

Les poids sont à nouveau mis à jour.

Rappel de la règle de mise à jour :

$$w_n^{(t)} = w_n^{(t-1)}(1 - \varepsilon \text{res}_n^{(t)}) \quad \text{avec} \quad \text{res}_n^{(t)} = \begin{cases} 0 & \text{si expert correct} \\ 1 & \text{sinon} \end{cases}$$

À l'instant $t = 2$:

- ✗ expert 1 : incorrect
- ✓ expert 2 : correct
- ✓ expert 3 : correct

Rappel de la règle de mise à jour :

$$w_n^{(t)} = w_n^{(t-1)} (1 - \varepsilon \text{res}_n^{(t)}) \quad \text{avec} \quad \text{res}_n^{(t)} = \begin{cases} 0 & \text{si expert correct} \\ 1 & \text{sinon} \end{cases}$$

À l'instant $t = 2$:

- ✗ expert 1 : incorrect
- ✓ expert 2 : correct
- ✓ expert 3 : correct

Donc :

$$w_1^{(2)} = (1 - \varepsilon) w_1^{(1)} = (1 - \varepsilon) w_1^{(0)}$$

$$w_2^{(2)} = w_2^{(1)} = w_2^{(0)}$$

$$w_3^{(2)} = w_3^{(1)} = (1 - \varepsilon) w_3^{(0)}$$

Interprétation

L'expert basé sur l'âge, le sexe et les comorbidités devient moins influent :
il s'est trompé sur ce patient.

Le Boosting adapte progressivement la confiance accordée à chaque expert.

Interprétation

L'expert basé sur l'âge, le sexe et les comorbidités devient moins influent :
il s'est trompé sur ce patient.

Le Boosting adapte progressivement la confiance accordée à chaque expert.

On comprend donc bien qu'au bout du t -ème client, on va avoir :

Interprétation

L'expert basé sur l'âge, le sexe et les comorbidités devient moins influent :
il s'est trompé sur ce patient.

Le Boosting adapte progressivement la confiance accordée à chaque expert.

On comprend donc bien qu'au bout du t -ème client, on va avoir :

Apprentissage récursif du Boosting

$$w_n^{(t)} = w_n^{(0)} (1 - \varepsilon)^{\sum_{s=1}^t \text{res}_n^{(s)}}$$

Interprétation

L'expert basé sur l'âge, le sexe et les comorbidités devient moins influent :
il s'est trompé sur ce patient.

Le Boosting adapte progressivement la confiance accordée à chaque expert.

On comprend donc bien qu'au bout du t -ème client, on va avoir :

Apprentissage récursif du Boosting

$$w_n^{(t)} = w_n^{(0)} (1 - \varepsilon)^{\sum_{s=1}^t \text{res}_n^{(s)}}$$

Le poids d'un expert dépend uniquement de son **nombre total d'erreurs cumulées** : plus un expert se trompe souvent, plus son influence diminue

Ainsi, le Boosting construit progressivement une **distribution adaptative de confiance** sur les experts.

- 1 Motivations
- 2 Illustration concrète
- 3 Algorithme du boosting**
- 4 Application : algorithme de Hedge
- 5 Boosting en classification

À l'instant t , on choisit un expert selon la loi :

$$p_n^{(t)} = \frac{w_n^{(t-1)}}{\sum_{i=1}^N w_i^{(t-1)}}$$

La perte moyenne est alors :

$$\mathbb{E}[\text{RES}^{(t)}] = \sum_{n=1}^N \text{res}_n^{(t)} \frac{w_n^{(t-1)}}{\sum_{i=1}^N w_i^{(t-1)}}$$

⇒ L'agrégateur réalise une moyenne pondérée des erreurs des experts.

Résultat

On a, pour tout $t \geq 1$,

$$\sum_{n=1}^N w_n^{(t)} = \left(\sum_{n=1}^N w_n^{(t-1)} \right) \left(1 - \varepsilon \mathbb{E}[\text{RES}^{(t)}] \right) \quad (\text{Eq1})$$

Démonstration.

Par définition de la mise à jour des poids,

$$w_n^{(t)} = w_n^{(t-1)} (1 - \varepsilon \text{res}_n^{(t)}).$$

En sommant sur $n = 1, \dots, N$, on obtient

$$\sum_{n=1}^N w_n^{(t)} = \sum_{n=1}^N w_n^{(t-1)} (1 - \varepsilon \text{res}_n^{(t)}).$$

En développant puis en factorisant par $\sum_{n=1}^N w_n^{(t-1)}$:

$$\sum_{n=1}^N w_n^{(t)} = \left(\sum_{n=1}^N w_n^{(t-1)} \right) \left(1 - \varepsilon \sum_{n=1}^N \text{res}_n^{(t)} \frac{w_n^{(t-1)}}{\sum_{i=1}^N w_i^{(t-1)}} \right).$$

Or, par définition de la loi de probabilité sur les experts à l'instant t ,

$$p_n^{(t-1)} = \frac{w_n^{(t-1)}}{\sum_{i=1}^N w_i^{(t-1)}},$$

et donc

$$\mathbb{E}[\text{RES}^{(t)}] = \sum_{n=1}^N \text{res}_n^{(t)} p_n^{(t-1)} = \sum_{n=1}^N \text{res}_n^{(t)} \frac{w_n^{(t-1)}}{\sum_{i=1}^N w_i^{(t-1)}}.$$

On obtient bien

$$\sum_{n=1}^N w_n^{(t)} = \left(\sum_{n=1}^N w_n^{(t-1)} \right) \left(1 - \varepsilon \mathbb{E}[\text{RES}^{(t)}] \right).$$

Puisque l'on a :

$$\sum_{n=1}^N w_n^{(t)} = \left(\sum_{n=1}^N w_n^{(t-1)} \right) \left(1 - \varepsilon \mathbb{E}[\text{RES}^{(t)}] \right),$$

en répétant ce raisonnement (par récurrence, ou “en cascade”), on établit l'Équation (Eq1) :

$$\sum_{n=1}^N w_n^{(t)} = N \prod_{s=1}^t \left(1 - \varepsilon \mathbb{E}[\text{RES}^{(s)}] \right)$$

Cette relation donne directement une correspondance

poids cumulés $\leftrightarrow$ erreur moyenne

Pourquoi cette relation est-elle importante ?

La quantité

$$W^{(t)} := \sum_{n=1}^N w_n^{(t)}$$

joue le rôle de **potentiel global** du système.

- si l'agrégateur commet peu d'erreurs en moyenne, alors les facteurs

$$1 - \varepsilon \mathbb{E}[\text{RES}^{(s)}]$$

restent proches de 1, et donc $W^{(t)}$ reste relativement grand ;

- si l'agrégateur commet beaucoup d'erreurs, alors ces facteurs deviennent plus petits, et $W^{(t)}$ décroît plus vite.

Autrement dit, la somme des poids non renormalisés **encode la qualité cumulative** de la stratégie d'agrégation.

Résultat

Soit l'expert meilleur celui qui a commis le moins d'erreurs sur les t premières étapes. Alors on a :

$$\sum_{n=1}^N w_n^{(t)} \geq (1 - \varepsilon)^{\sum_{s=1}^t \text{res}_{\text{meilleur}}^{(s)}} \quad (\text{InEq1})$$

Interprétation

La somme totale des poids est nécessairement supérieure ou égale au poids de n'importe quel expert pris individuellement, et donc en particulier à celui du meilleur expert.

Cette inégalité fournit une **borne inférieure** sur le potentiel global

$$W^{(t)} = \sum_{n=1}^N w_n^{(t)},$$

qui sera ensuite comparée à la borne supérieure obtenue via l'erreur moyenne de l'agrégateur.

Étape 1 : la somme des poids est toujours supérieure ou égale au poids de n'importe quel expert. En particulier,

$$\sum_{n=1}^N w_n^{(t)} \geq w_{\text{meilleur}}^{(t)}.$$

Étape 2 : on explicite le poids du meilleur expert à l'instant t .

Par la règle de mise à jour,

$$w_n^{(t)} = w_n^{(0)} (1 - \varepsilon)^{\sum_{s=1}^t \text{res}_n^{(s)}}.$$

Donc, pour l'expert meilleur,

$$w_{\text{meilleur}}^{(t)} = w_{\text{meilleur}}^{(0)} (1 - \varepsilon)^{\sum_{s=1}^t \text{res}_{\text{meilleur}}^{(s)}}.$$

Étape 3 : on utilise l'initialisation uniforme.

Au départ, tous les experts ont poids 1, donc $w_{\text{meilleur}}^{(0)} = 1$. Par conséquent,

$$w_{\text{meilleur}}^{(t)} = (1 - \varepsilon)^{\sum_{s=1}^t \text{res}_{\text{meilleur}}^{(s)}}.$$

Conclusion :

$$\sum_{n=1}^N w_n^{(t)} \geq w_{\text{meilleur}}^{(t)} = (1 - \varepsilon)^{\sum_{s=1}^t \text{res}_{\text{meilleur}}^{(s)}}.$$

C'est exactement l'inégalité annoncée.

On combine l'Inéquation (InEq1) avec l'Équation (Eq1).

Puisque :

$$\sum_{n=1}^N w_n^{(t)} \geq (1 - \varepsilon)^{\sum_{s=1}^t \text{res}_{\text{meilleur}}^{(s)}}$$

On établit donc l'encadrement suivant :

Résultat

$$N \prod_{s=1}^t \left(1 - \varepsilon \mathbb{E}[\text{RES}^{(s)}]\right) \geq (1 - \varepsilon)^{\sum_{s=1}^t \text{res}_{\text{meilleur}}^{(s)}} \quad (\text{Encad1})$$

On compare ainsi directement

- la performance de l'agrégateur
- celle du meilleur expert

Après quelques opérations laissées au bon plaisir du lecteur, appliquées à l'encadrement (Encad1), on obtient finalement la borne :

Encadrement fondamental sur le risque

$$\frac{1}{t} \sum_{s=1}^t \mathbb{E}[\text{RES}^{(s)}] \leq \frac{1}{t} \sum_{s=1}^t \text{res}_{\text{meilleur}}^{(s)} + \sqrt{\frac{\log(N)}{t}}$$

Après quelques opérations laissées au bon plaisir du lecteur, appliquées à l'encadrement (Encad1), on obtient finalement la borne :

Encadrement fondamental sur le risque

$$\frac{1}{t} \sum_{s=1}^t \mathbb{E}[\text{RES}^{(s)}] \leq \frac{1}{t} \sum_{s=1}^t \text{res}_{\text{meilleur}}^{(s)} + \sqrt{\frac{\log(N)}{t}}$$

Donc :

- ⇒ L'agrégateur est presque aussi performant que le meilleur expert (*sans connaître a priori ce meilleur expert !*);
- ⇒ Le tout avec un écart qui décroît lorsque t augmente.



Intérêt du Boosting

Le Boosting permet de construire un prédicteur global dont la performance est proche du meilleur expert disponible, avec un écart contrôlé :

$$\sqrt{\frac{\log(N)}{t}}$$

Ce terme :

- ✓ dépend faiblement du nombre d'experts
- ✓ décroît avec le temps

Le Boosting constitue ainsi une méthode robuste et théoriquement garantie pour agréger plusieurs modèles faibles.

- 1 Motivations
- 2 Illustration concrète
- 3 Algorithme du boosting
- 4 Application : algorithme de Hedge**
- 5 Boosting en classification

Algorithme

On considère N experts et un horizon temporel T . À chaque date $t = 1, \dots, T$:

1. on dispose de poids positifs $w_{t,1}, \dots, w_{t,N} > 0$.
2. on définit une distribution de probabilité

$$p_{t,i} = \frac{w_{t,i}}{\sum_{j=1}^N w_{t,j}}$$

3. après observation des pertes $\ell_{t,1}, \dots, \ell_{t,N} \in [0, 1]$, on met à jour les poids

$$w_{t+1,i} \leftarrow w_{t,i} e^{-\eta \ell_{t,i}}$$

où $\eta > 0$ est le paramètre d'apprentissage

La perte attendue du joueur à la date t vaut

$$\hat{\ell}_t = \sum_{i=1}^N p_{t,i} \ell_{t,i}.$$

On considère : $N = 3$, $T = 3$ Initialisation : $w_{1,1} = w_{1,2} = w_{1,3} = 1$ et $\eta = \ln 2$. Les pertes observées sont :

t	$\ell_{t,1}$	$\ell_{t,2}$	$\ell_{t,3}$
1	0	1	0
2	1	0	1
3	0	1	1

Questions

1. Calculer les distributions p_t
2. Calculer les poids successifs w_2, w_3, w_4
3. Calculer les pertes attendues $\hat{\ell}_t$
4. Calculer la perte cumulée de Hedge
5. Comparer avec le meilleur expert fixe

Initialisation :

$$w_1 = (1, 1, 1)$$

Donc

$$p_1 = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$$

Les pertes sont :

$$(\ell_{1,1}, \ell_{1,2}, \ell_{1,3}) = (0, 1, 0)$$

Perte attendue :

$$\hat{\ell}_1 = \frac{1}{3}$$

Comme

$$\eta = \ln 2 \quad \Rightarrow \quad e^{-\eta} = \frac{1}{2}$$

on obtient

$$w_2 = \left(1, \frac{1}{2}, 1\right)$$

Normalisation :

$$p_2 = \left(\frac{2}{5}, \frac{1}{5}, \frac{2}{5} \right)$$

Pertes observées :

$$(1, 0, 1)$$

Perte attendue :

$$\widehat{\ell}_2 = \frac{4}{5}$$

Mise à jour :

$$w_3 = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2} \right)$$

Comme les poids sont identiques :

$$p_3 = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$$

Pertes observées :

$$(0, 1, 1)$$

Perte attendue :

$$\widehat{\ell}_3 = \frac{2}{3}$$

Mise à jour finale :

$$w_4 = \left(\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right)$$

Perte cumulée de Hedge :

$$\widehat{L}_T = \widehat{\ell}_1 + \widehat{\ell}_2 + \widehat{\ell}_3 = \frac{9}{5}$$

Pertes cumulées des experts :

$$L_1 = 1 \quad L_2 = 2 \quad L_3 = 2$$

Le meilleur expert fixe est :

$$\min_i L_i = 1$$

Regret :

$$R_T = \widehat{L}_T - \min_i L_i = \frac{4}{5}$$

Hedge ne suit pas exactement le meilleur expert, mais reste proche de sa performance.

Cet exercice illustre trois idées fondamentales :

- ✓ les experts performants conservent un poids élevé
- ✓ les experts mauvais sont pénalisés exponentiellement
- ✓ l'agrégateur reste proche du meilleur expert fixe

Résultat théorique classique :

$$R_T \leq \frac{\ln N}{\eta} + \frac{\eta T}{8}$$

pour des pertes dans $[0, 1]$.

- 1 Motivations
- 2 Illustration concrète
- 3 Algorithme du boosting
- 4 Application : algorithme de Hedge
- 5 Boosting en classification**

Exemple concret

On souhaite prédire si un patient est **à risque** ou **non à risque**.

On reprend notre exemple d'outils de décision simples :

1. un arbre basé sur l'âge, le sexe et les comorbidités
2. une règle basée sur la tension artérielle
3. un score basé sur des biomarqueurs

Chaque outil donne une prédiction partielle. *Comment combiner ces prédictions pour obtenir une décision fiable ?*

Motivation

Construire un **classifieur fort** à partir d'une combinaison séquentielle de **classifieurs faibles**.

Principe

- chaque classifieur faible est légèrement meilleur que le hasard
- leurs sorties sont combinées avec des poids adaptatifs
- les observations mal classées deviennent progressivement prioritaires

Idée naturelle

- chaque classifieur faible prédit $+1$ ou -1
- on pourrait naïvement additionner ces prédictions

$$F(x) = \sum_{m=1}^M g_m(x)$$

- puis décider selon le signe de cette somme (vote majoritaire)

Boosting

Le Boosting améliore cette idée :

$$F_M(x) = \sum_{m=1}^M \alpha_m g_m(x)$$

où chaque coefficient α_m pondère la contribution du classifieur selon sa qualité.

On suppose :

$$y_i \in \{-1, 1\}, \quad g_m(x) \in \{-1, 1\}$$

Hypothèse clé

Chaque classifieur faible vérifie :

$$\mathbb{P}(g_m(x) = y) > 0.5 \iff e_m < 0.5$$

c'est-à-dire qu'il est légèrement meilleur que le hasard. *En gros, on mesure la probabilité que la prédiction par le modèle g_m soit bien la bonne.*

⇒ le Boosting peut exploiter cette information pour construire un classifieur global performant.

On construit séquentiellement une suite de classifieurs faibles

$$g_1, \dots, g_M$$

où

M = nombre total d'itérations de boosting.

À chaque étape m :

1. On apprend un classifieur faible g_m sur les données pondérées :

$$g_m = \arg \min_g \sum_{i=1}^n w_i^{(m)} \mathbb{1}_{\{y_i \neq g(x_i)\}}$$

Les observations difficiles ont plus de poids.

2. On mesure son erreur pondérée :

$$e_m = \frac{\sum w_i^{(m)} \mathbb{1}_{\{y_i \neq g_m(x_i)\}}}{\sum w_i^{(m)}} \quad (\text{Erreur-}m)$$

3. On calcule son importance dans la décision finale :

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - e_m}{e_m} \right) \quad (\text{Pond-}\alpha)$$

4. On augmente le poids des observations mal classées :

$$w_i^{(m+1)} = w_i^{(m)} \exp \left(\alpha_m \mathbb{1}_{\{y_i \neq g_m(x_i)\}} \right)$$

Ainsi, le classifieur suivant se concentre davantage sur ces observations.

Après M itérations :

$$F_M(x) = \sum_{m=1}^M \alpha_m g_m(x)$$

Décision :

$$\hat{g}(x) = \text{sign}(F_M(x))$$

Interprétation

Chaque classifieur vote :

- ✓ vote pondéré par sa qualité
- ✓ les meilleurs classifieurs dominent la décision finale

Exemple concret

On souhaite prédire si un patient est **à risque** ou **non à risque**.

On dispose de trois outils simples :

1. un arbre basé sur l'âge, le sexe et les comorbidités
2. une règle basée sur la tension artérielle
3. un score basé sur des biomarqueurs

Notation importante :

- m désigne le numéro de l'itération de Boosting
- $g_m(x)$ désigne le classifieur faible appris à l'étape m
- M désigne le nombre total d'itérations

Initialisation ($m = 1$)

$$w_i^{(1)} = \frac{1}{n}$$

Toutes les observations ont le même poids :
aucune observation n'est prioritaire.

On choisit alors le premier outil de décision :

$$g_1(x) = \text{arbre \text{âge-sexe-comorbidités}}$$

Certaines observations sont mal classées par g_1 :
leur poids augmente pour l'étape suivante.

Deuxième itération ($m = 2$)

Les observations mal classées deviennent plus influentes :

$$w_i^{(2)} = w_i^{(1)} \exp(\alpha_1 \mathbb{1}_{\{y_i \neq g_1(x_i)\}})$$

On sélectionne à l'étape $m = 2$ le modèle qui corrige au mieux les erreurs du premier classifieur.

$$g_m = \arg \min_{g \in \mathcal{G}} \sum_{i=1}^n w_i^{(2)} \mathbb{1}_{y_i \neq g(x_i)}$$

où $\mathcal{G}$ est notre bibliothèque de modèles simples à disposition. Disons par exemple qu'on obtienne

$$g_2(x) = \text{règle basée sur la tension artérielle}$$

qui se concentre davantage sur ces observations difficiles.

Chaque nouveau modèle corrige prioritairement les erreurs du précédent.

Après deux étapes :

$$F_2(x) = \alpha_1 g_1(x) + \alpha_2 g_2(x)$$

où les coefficients α_1 et α_2 mesurent la qualité des classifieurs g_1 et g_2 et sont obtenus par (Erreur- m) puis (Pond- α).

Modèle additif

Plus généralement :

$$F_M(x) = \sum_{m=1}^M \alpha_m g_m(x)$$

Boosting construit progressivement un classifieur fort en corrigeant séquentiellement les erreurs précédentes.

Exponential loss

AdaBoost minimise la fonction de coût :

$$L(y, f(x)) = e^{-yf(x)}$$

appelée **exponential loss**.

Erreur empirique :

$$E_m = \frac{1}{n} \sum_{i=1}^n e^{-y_i F_m(x_i)}$$

Lecture

AdaBoost construit séquentiellement

$$F_m(x) = \sum_{k=1}^m \alpha_k g_k(x)$$

en minimisant cette erreur.

On pose :

$$w_i^{(m)} = \exp(-y_i F_{m-1}(x_i))$$

Alors :

$$E_m = \frac{1}{n} \sum_{i=1}^n w_i^{(m)} e^{-\alpha_m y_i g_m(x_i)}$$

Minimiser cette quantité donne justement la formule de l'Équation (Pond- α) :

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - e_m}{e_m} \right)$$

Pour la culture : Sur le plan statistique, cette quantité correspond à une estimation du log-rapport de vraisemblance (log-odds) associé à la capacité du classifieur à discriminer correctement les deux classes. Plus ce rapport est élevé, plus le classifieur apporte une information discriminante forte sur la classe du patient.

Conclusion

Plus un classifieur est précis,
plus son poids dans la décision finale est élevé.

AdaBoost est un cas particulier du cadre de [Gradient Boosting](#).

Principe général :

$$\min_F \sum_{i=1}^n L(y_i, F(x_i))$$

AdaBoost correspond au cas :

$$L(y, f) = e^{-yf}$$

Interprétation moderne

Boosting = descente de gradient dans un espace de fonctions.

Intérêt de la méthode

AdaBoost améliore progressivement la performance du classifieur global :

- en augmentant le poids des observations difficiles
- en pondérant les classifieurs selon leur qualité
- en construisant un modèle additif robuste

Il constitue l'un des fondements théoriques du

Gradient Boosting moderne

(XGBoost, LightGBM, CatBoost).

Données

On considère $n = 4$ observations (x_i, y_i) avec $y_i \in \{-1, +1\}$ (*pas besoin de préciser les x_i pour cet exemple illustratif*):

$$(y_1, y_2, y_3, y_4) = (+1, +1, -1, -1)$$

Initialisation des poids :

$$w_i^{(1)} = \frac{1}{4}$$

Étape $m = 1$ **Étape $m = 1$: apprentissage du premier classifieur faible g_1**

Supposons que g_1 donne : $g_1(x) = (+1, -1, -1, -1)$.

Erreur pondérée :

$$e_1 = \sum_i w_i^{(1)} \mathbb{1}_{y_i \neq g_1(x_i)} = \frac{1}{4}$$

Poids du classifieur :

$$\alpha_1 = \frac{1}{2} \log \left(\frac{1 - e_1}{e_1} \right) = \frac{1}{2} \log(3)$$

Mise à jour des poids :

$$w_i^{(2)} = w_i^{(1)} \exp(\alpha_1 \mathbf{1}_{y_i \neq g_1(x_i)})$$

Après normalisation : $w^{(2)} \approx (0.21, 0.37, 0.21, 0.21)$. L'observation mal classée devient prioritaire.

Étape $m = 2$

Étape $m = 2$: apprentissage du second classifieur g_2

Supposons :

$$g_2(x) = (+1, +1, +1, -1)$$

Vérifiez le(s) observation(s) mal classée(s). Le calcul de l'erreur pondérée donne

$$e_2 = 0.21$$

Poids du classifieur :

$$\alpha_2 = \frac{1}{2} \log \left(\frac{1 - e_2}{e_2} \right) \approx 0.66$$

Classifieur final AdaBoost

$$F(x) = \alpha_1 g_1(x) + \alpha_2 g_2(x)$$

Décision :

$$\hat{y} = \text{sign}(F(x))$$

Avec ce modèle, on a comme prédiction pour l'observation 3 : $g_1 = -1$, $g_2 = +1$ Donc :

$$F(x_3) = 0.55(-1) + 0.66(+1) = +0.11$$

Donc prédiction finale : +1

L'erreur est bien corrigée ! L'observation est bien "prédite".

Interprétation

AdaBoost construit progressivement un classifieur fort :

$$F_M(x) = \sum_{m=1}^M \alpha_m g_m(x)$$

où chaque nouveau modèle corrige prioritairement les observations mal classées par les précédents.